DrowsiGuard: Real-Time Drowsiness Detection System

PROJECT REPORT

COMPUTER VISION (CSE3010)

Slot: B14 + D21

Class No.: BL2025260500501

Submitted by:
Name: Divyansh Kumar
Reg. no.: 23BAI10514

Submitted to:
Dr. Shahana Gazala Qureshi

1. Problem Definition

1.1 The Problem I chose to solve

Driver fatigue and microsleeps kill people on the road. Unlike speeding or reckless driving, these problems are difficult for drivers to self-diagnose, not typically malicious, and ultimately prove fatal if nothing is done to intervene. I chose to develop DrowsiGuard in order to detect the precise point in time that a driver's mental functions deteriorate, and their eyelids naturally begin to shut.

1.2 Why this matters

According to global road safety statistics, approximately 20% of serious vehicle accidents are caused by fatigue. Typical safety features, like lane departure warnings, react once the car begins to drift; this project addresses the problem by proactively determining whether the driver is biologically suffering from drowsiness before it causes the car to stray from its intended path.

2. How I approached this problem

Computer Vision (CV) was chosen over a wearable system (like a heart monitor) because a camera is an intrusive, low-cost solution that drivers are more likely to accept. My approach relies on Facial Landmark Detection in order to pinpoint the eyes. These facial landmarks are then analyzed through mathematical operations (instead of using a resource-intensive deep learning model) to determine if the eyes are showing signs of drowsiness. A mathematical operation was selected over a deep learning one because the goal was to make this solution readily usable on any standard laptop or low-powered embedded system (like a Raspberry Pi).

3. Technical Methodology

3.1 Eye Aspect Ratio (EAR) Algorithm

To quantify what "closed eyes" means, I utilized the EAR (Eye Aspect Ratio) algorithm. 6 (x,y) coordinates for the eyes are identified by the system:

$$EAR = \frac{\| p_2 - p_6 \| + \| p_3 - p_5 \|}{2 \cdot \| p_1 - p_4 \|}$$

Vertical distances: $\| p_2 - p_6 \|$ and $\| p_3 - p_5 \|$
Horizontal distance: $\| p_1 - p_4 \|$

High and stable EAR signifies open eyes. Closed eyes result in a sharp dip of EAR towards zero due to a decreasing numerator.

3.2 Detection Pipeline

- Camera captures 30 FPS video, efficiently webcams.
- Frames are then preprocessed into a 450-pixel width, and then the color is removed to minimize the computational power, making processing much more efficient.
- A HOG + Linear SVM algorithm built into Dlib is used to detect the driver's face.
- A 68-point predictor is applied to isolate the eyes ([36-48] points on the face are detected).
- A state machine tracks EAR. If the value drops below 0.25 for 20 consecutive frames, an alarm is issued.

4. Key decisions I made

EAR vs. CNNs: I purposefully decided against utilizing a CNN (Convolutional Neural Network) because it would require an incredibly powerful GPU to achieve 30+ FPS. With the EAR algorithm, a high frame rate can be achieved on any standard CPU, while only requiring about 150 MB of memory.

The "Persistence" buffer: An essential part of my design was the frame counter of 20 frames; this was important in distinguishing normal blinking from real drowsiness.
Grayscale Operation: I operated on grayscale images in order to minimize the amount of data that needed to be processed on each frame by 2/3rds of the original size, effectively lowering latency.

5. Challenges and Technical results

5.1 Challenges I encountered

Jitter: Lower-quality web cameras caused the landmarks to jump slightly, giving false positives on "closed eyes."
Its Solution: I added a Circular Buffer which takes the EAR values over 5 frames, computing an average so that jittery values have less effect.

Lighting: In the dark, the camera could not effectively detect facial landmarks.
Its Solution: This project helped me understand that software isn't enough to completely combat a hardware issue, and IR LEDs would be required in order to create a product that drivers could use in any lighting condition.

5.2 Performance metrics

METRIC	RESULT
Average processing speed	28 - 32 FPS (Mid-range CPU)
Detection accuracy (daylight)	~96%
Detection accuracy (low light)	~78% (Requires IR-sensitive camera)
False positive rate (blinks)	< 2% (due to frame persistence logic)

6. What I learned

From this project, I learned how to bridge the gap between pixel-level computer vision data and human physiological states such as sleepiness.
Specifically, I learned about the computational limits of real-time processing, the importance of balancing math models with hardware requirements, and how to filter data to reduce false positives.

7. Conclusion and future work

DrowsiGuard demonstrates that powerful automotive safety systems do not need to rely on heavy, expensive hardware in order to be effective.

I intend to further develop this project into a multi-modal system by implementing yawn detection using the Mouth Aspect Ratio (MAR), as well as estimating head pose in order to determine if the driver is too distracted by something on their person or by a mobile device.

I plan on integrating a notification system that can send alerts to fleet managers for commercial trucking uses, further expanding the capabilities and safety impact of the DrowsiGuard system.